多层感知器：隐藏层、反向传播与梯度下降

读前须知

- **前置**：链式法则、梯度下降；如果对矩阵求导不熟，可以先跳过推导，只记住结论的形状。
- **这一篇的位置**：现代神经网络的地基。所有深度模型的反向传播，都是这一篇那几条链式法则的重复。
- **读完能做什么**：写出单隐藏层网络的前向计算；手推一层权重的梯度；解释隐藏层为什么能解开异或，以及梯度消失从哪里来。

一、用多层结构破解异或

1.1 一个能算异或的网络

虽然异或问题成为感知器和早期神经网络的阿喀琉斯之踵，但它并非无解的问题。恰恰相反，解决它的思路相当简单，就是将单层感知器变成多层感知器。

在输入层与输出层之间插入一层神经元，异或就能被算出来。考虑两个输入节点 A 和 B，中间放两个隐藏神经元 C 和 D，最后接一个输出神经元 E。取这样一组权重：

- C 的输入为 A + B − 0.5，阈值取 0，即 C 在 A、B 至少一个为 1 时激活——这相当于“或”；
- D 的输入为 A + B − 1.5，阈值取 0，即 D 只在 A、B 同时为 1 时激活——这相当于“与”；
- E 的输入为 C − D − 0.5，阈值取 0。

逐个验算四种输入：

A	B	C 的输入	C 输出	D 的输入	D 输出	E 的输入	E 输出
0	0	−0.5	0	−1.5	0	−0.5	0
0	1	0.5	1	−0.5	0	0.5	1
1	0	0.5	1	−0.5	0	0.5	1
1	1	1.5	1	0.5	1	−0.5	0

输出列恰好就是异或的真值表。这个网络之所以能行，是因为“或”与“与”这两个中间量把原本不可分的问题拆成了两步：先用两个线性函数划出两个半空间，再用一个线性函数对它们的输出做组合。

1.2 多层感知器的结构

多层感知器（multilayer perceptron）包含一个或多个在输入节点和输出节点之间的隐藏层（hidden layer），除了输入节点外，每个节点都是使用非线性激活函数的神经元。

而在不同层之间，多层感知器具有全连接性，即任意层中的每个神经元都与它前一层中的所有神经元或者节点相连接，连接的强度由网络中的权重系数决定。全连接的含义是：这一层没有“局部”的概念，每个单元都看得到上一层的全部输出。

1.3 前馈网络

多层感知器是一类**前馈人工神经网络**（feedforward neural network）。网络中每一层神经元的输出都指向输出方向，也就是向前馈送到下一层，直到获得整个网络的输出为止。

“前馈”这个限定很重要：它排除了层与层之间有回路的结构。没有回路意味着给定输入后，输出可以在一次前向扫描中全部算完，不需要迭代到稳定状态。这正是反向传播能够用一次链式法则求完所有梯度的前提。

补充说明：上面那组异或权重并不唯一，把所有权重同时放大若干倍，输出完全不变，因为符号函数只关心正负。这暴露出一个更深的事实：单层感知器的解有无穷多个，而多层的解空间更大。网络能表达什么，与网络实际学到什么，是两个不同的问题——前者由结构决定，后者由训练数据和优化过程决定。训练之所以困难，很大程度上就是因为同一个结构下有太多组权重都能拟合训练集，而它们的泛化能力可能相差很远。

二、误差函数：定义“学得好不好”

2.1 为什么不能直接相减

多层感知器的训练包括以下步骤：首先确定给定输入和当前权重下的输出，再将输出和真实值相减得到误差函数，最后根据误差函数更新权重。

当多层感知器具有多个输出时，每个分类结果 y_j 与真实结果 d_j 之间都会存在误差。在单层感知器中，误差直接被定义为两者之间的差值。但在多个输出的情形下，如果第一个输出神经元的误差大于零，第二个输出神经元的误差小于零，这两部分误差就可能部分甚至完全抵消，造成分类结果准确无误的假象。

2.2 平方误差

为了避免这个问题，在反向传播算法中，每个输出神经元的误差都被写成平方项的形式，整个神经网络的误差则是所有输出神经元的误差之和：

$$E = \frac{1}{2} \sum_j (d_j - y_j)^2$$

如此一来，误差函数就以二次型的形式体现，也就避免了符号的影响。系数 1/2 是为了求导时把平方产生的因子 2 约掉，让梯度表达式更简洁。

明确定义了误差函数后，就要想方设法让它取得最小值。影响误差函数的因素无外乎三个：输入信号、传递函数和权重系数。输入信号是完全不依赖于神经网络的外部信号，无法更改；传递函数在网络设计过程中已经确定，同样无法更改；所以在算法执行的过程中，能够更新的就只有权重系数了。

补充说明：平方误差不是随手选的。若假设观测值等于模型输出加上一个零均值的高斯噪声，那么最大化样本的似然函数等价于最小化平方误差之和。也就是说，平方误差对应的是“噪声服从高斯分布”这一假设。如果噪声的分布重尾（离群点较多），平方误差会被极端值主导，此时改用绝对误差更稳健。这个对应关系在广义线性模型的框架里会反复出现。

三、反向传播：误差的逆向流动

3.1 从输出层回溯

在训练过程中，虽然信号的流向是输出方向，但计算出的误差函数和信号传播的方向相反，也就是向输入方向传播的，正因如此，这种学习方式得名**反向传播**（backpropagation）。

反向传播算法通过求解误差函数关于每个权重系数的偏导数，以此使误差最小化来训练整个网络。

为什么必须反向？因为误差只在输出层被定义。隐藏层的神经元没有“正确答案”，它们的输出是对是错无法直接判断。唯一可行的办法是：先算出输出层的误差，再把这份误差按权重比例分摊给每个隐藏神经元，逐层往回推。

3.2 链式法则

既然提到了传递函数，就有必要对多层感知器中的传递函数加以说明。单层感知器中使用的符号函数有一个缺点：它是不连续的函数，因而不能在不连续点上求解微分。为了解决这个弊端，多层感知器采用对数几率函数作为传递函数——也就是逻辑回归里用过的那个 Sigmoid 函数。

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

它的导数有一个非常简洁的形式：

$$\sigma'(z) = \sigma(z)\,(1 - \sigma(z))$$

根据最优化理论，求解误差函数的最小值就要找到误差函数的梯度，再根据梯度调整权重系数。对误差函数的求解从输出节点开始，通过神经网络逆向传播，直到回溯到输入节点。

这背后的原理在于权重系数的变化对输出的影响方式并非直接修改，而是通过隐藏层逐渐扩散。这样环环相扣的作用方式体现在数学上就是求导的链式法则。链式法则的思想是求解从权重系数到误差函数这个链条上每一环的作用，再将每一环的作用相乘，得到的就是链条整体的效果。

3.3 权重更新公式

把链式法则落到具体形式上。对输出层的权重 w_{hi}，误差梯度可以写成

$$\frac{\partial E}{\partial w_{hi}} = -\,(d_i - y_i)\sigma'(\mathrm{net}_i)z_h$$

式中 z_h 是隐藏神经元的输出，net_i 是输出神经元的加权输入。把前面三项合并记作**局域梯度** δ_i，则有

$$\delta_i = (y_i - d_i)\sigma'(\mathrm{net}_i), \qquad \Delta w_{hi} = -\eta\,\frac{\partial E}{\partial w_{hi}} = \eta\,\delta_i\,z_h$$

隐藏层的局域梯度则由上一层所有局域梯度的加权和算出：

$$\delta_h = \sigma'(\mathrm{net}_h)\sum_i w_{hi}\,\delta_i$$

于是整个算法可以概括为四步：初始化网络中所有权重系数和阈值；在前向计算中将训练样本送入输入节点，在输出节点得到结果，再以平方误差形式计算误差函数；在反向计算中从输出层到隐藏层逐层计算局域梯度，并按 $\Delta w = -\eta \partial E/\partial w$ 更新权重；用新样本迭代前向与反向过程，直到满足停止准则。

> **新手提示：为什么要随机初始化权重**
>
> 若所有权重都初始化为同一个值，同一层的所有神经元在每次更新中会得到完全相同的梯度，从而始终保持相同的权重。这样的网络等价于每层只有一个神经元，隐藏层就白设了。随机初始化打破这种对称性，让不同神经元从不同的起点出发，逐渐分化出不同的功能。对称性破缺是深度网络能训练起来的前提条件之一。

四、为什么传递函数必须可微

4.1 符号函数的缺陷

抛开冗杂的数学符号和运算不论，反向传播算法的原理其实并没有那么复杂。如果你能在头脑中勾勒出包含单个隐藏层的多层感知器结构，不妨直观想象一下当某个输入上出现细微的扰动会导致什么结果。

由于层与层之间的连接方式是全连接，单个输入的微小变化会传递到所有的隐藏神经元。由于隐藏神经元的输入是输入信号的线性组合，输入端的扰动体现在隐藏神经元上就经过了一重权重系数的放大。经过放大后的扰动又被送入非线性的传递函数里，传递函数输入端的扰动导致的输出端改变，还要再乘以一个传递函数的导数作为放大因子。

整条链路上，每一环的放大倍数都是可计算的，但前提是每一环都能求导。符号函数在阈值处发生跳变，导数在那里没有定义，在其他地方导数恒为零——这意味着梯度要么不存在，要么不携带任何信息，误差根本无法回传。

4.2 对数几率函数

Sigmoid 函数正好补上这个缺口：它处处可导，导数连续，而且把任意实数压缩到 (0, 1) 区间，输出可以解释为概率。它的另一个好处是导数可以用自身表示，计算一次前向就能顺手算出反向所需的导数，不必重复求值。

补充说明：Sigmoid 的导数最大只有 0.25，而且在两端迅速趋近于零。反向传播时每一层都要乘上一个这样的导数，层数一多，梯度就会以指数速度衰减，靠近输入层的权重几乎收不到有效的修正信号——这就是**梯度消失**。它解释了为什么早期多层网络难以训练，深度一增加就学不动。后来的解法包括改用 ReLU 这类正区间导数恒为 1 的激活函数、引入残差连接让梯度有短路通道、以及用 Batch Normalization 稳定各层输入分布。这个问题至今没有在生物学意义上被"解决"，只是被工程手段绕开了。

五、隐藏层在做什么

5.1 特征检测

多层感知器的核心结构就是隐藏层，之所以被称为隐藏层是因为这些神经元并不属于网络的输入或输出。

在多层神经网络中，隐藏神经元的作用在于特征检测。随着学习过程的不断进行，隐藏神经元将训练数据变换到新的特征空间之上，并逐渐识别出训练数据的突出特征。

回到异或的例子，"或"和"与"就是两个被构造出来的新特征。原始坐标下四个点混在一起分不开，但换到这个二维特征空间里，两类点就变成线性可分的了。隐藏层做的正是这样一件事：它不是在下判断，而是在换坐标系。

5.2 特征空间的可视化

从几何上看，每一层都在对上一层的表示做一次仿射变换加一次非线性挤压。仿射变换负责旋转、缩放、平移，非线性挤压负责把弯曲的边界"拉直"。层数越多，能拉直的弯曲越复杂。

补充说明：层数增加带来的不只是"更复杂"，还有"更省参数"。同一个函数若用单隐层网络逼近，隐层宽度可能要随目标函数的振荡次数指数增长；而用深层结构，每一层只需在前一层的基础上做一个简单的变换，参数量随层数线性增长。这背后的原因是层次化复用：低层学到的局部模式可以被高层反复调用，不必为每一种组合单独准备一个神经元。这也解释了为什么图像、语音、语言这类具有层级结构的数据特别适合深层网络——它们的生成过程本身就是分层的。

> **新手提示：隐藏层不是黑箱**

> 说隐藏层“不可解释”，指的是我们无法用自然语言给每个隐藏单元贴一个标签，而不是说它的行为无法分析。对任意一个隐藏单元，你都可以算出它对输入的响应模式，找到让它在训练集上激活最强的样本，从而大致判断它在检测什么。可解释性研究做的就是这件事。所谓黑箱，更多是“标注成本高”的问题，而非原理上的不可知。

六、网络设计的工程因素

6.1 参数数量与样本量

在解决实际问题时，多层感知器的设计要考虑一些工程因素。假设单隐层的多层感知器有 L 个输入节点、M 个隐藏节点和 N 个输出节点，那这个网络的权重系数总数就是

$$(L+1)\times M+(M+1)\times N$$

式中的 +1 来自每个神经元各有一个偏置。这些权重的取值都需要由反向传播算法确定，而反向传播算法又由训练数据的错误来驱动。因而用于训练的数据越多，多层感知器的学习效果也就越好。

一个经验法则是训练样本数目应该是权重系数数目的 10 倍，这显然对计算能力提出了较高的要求。从这个角度看，明斯基对感知器的批评是站得住脚的——在 1960 年代的硬件条件下，训练一个参数稍多的网络确实不现实。

6.2 隐藏层数与万能逼近

抛开训练数据量不论，隐藏层和隐藏神经元的数目也是网络设计中需要考虑的问题。

在数学上可以证明，单个隐藏层就能够实现凸区域，双隐藏层更是可以实现任意形状的区域（包括非凸的），也就能够解决任何复杂的分类问题。在隐藏层数目不变的前提下，区域的复杂程度还能够随着隐藏神经元数目的增加而提升。

在数学上同样可以证明，只要隐藏神经元的数目足够多，一个隐藏层就能使多层感知器以任意精度逼近任意复杂度的连续函数。这就是**万能逼近定理**。

通常情况下，多层感知器不会选择两个以上的隐藏层，因为层数越多，要追踪哪些权重正在被更新就越困难，又不会带来性能的提升。

6.3 迭代终止条件

多层感知器的训练需要多次遍历整个数据集，因而迭代次数就成为另一个重要的问题。预先设定迭代次数无法保证训练效果，预先设定误差阈值则可能导致算法无法终止。因而常用的办法是：一旦误差函数停止减小，就终止学习算法。

补充说明：万能逼近定理说的是“存在一组权重能逼近到任意精度”，它并没有说“梯度下降能找到这组权重”，更没有说“这样的网络能泛化”。这两个缺口后来成了深度学习的两大主题：前者靠更好的优化算法与初始化策略，后者靠正则化与海量数据。定理的工程意义因此要打折来看——它保证的是表达力的上限，不是可训练性与泛化能力的下限。另外，后来的研究表明深度带来的效率优势是真实的：同样的函数，深层网络所需的参数可能远少于浅层网络，这也是网络越做越深的理论依据之一。

七、过拟合与早停

同其他机器学习方法一样，多层感知器也面临过拟合的风险。模型的泛化能力可以通过验证集来监督，也就能够在一定程度上避免过拟合的发生。当训练集的误差下降但验证集的误差上升时让训练立即停止，这就是所谓**早停**的过拟合抑制策略。

早停之所以有效，是因为它把“训练多久”也变成了一个需要调节的超参数。训练轮数本身就是一个容量控制量：轮数太少模型欠拟合，轮数太多模型开始记忆训练集中的噪声。用验证误差做裁判，相当于在容量与拟合之间找一个折中点。

当然，正则化方法也可以应用在多层感知器的训练中。在误差函数上加上权重平方和，就得到 L2 正则；加上权重绝对值之和，就得到 L1 正则，后者还能顺带把一部分权重压到零，起到特征选择的作用。

补充说明：早停与 L2 正则之间有相当紧密的联系。若把权重初值取得很小、学习率也取得很小，梯度下降在训练早期对权重的调整幅度有限，效果上相当于把权重约束在一个小范数球内，而 L2 正则做的正是这件事。可以证明在某些简化条件下，早停等价于对权重范数施加惩罚。这解释了为什么两种手段常常能互相替代：它们约束的是同一个量，只是一个从迭代次数入手，一个从损失函数入手。

四个常见误区

- **“反向传播是一种学习算法”**——它只是计算梯度的方法，本身不规定怎么更新参数。真正决定学什么的，是损失函数、数据与优化器的选择。
- **“隐藏层越多越好”**——没有残差连接、归一化与足够数据时，加深网络反而会退化：训练误差都可能上升。
- **“万能逼近定理保证网络能学到”**——定理只说明存在一组权重可以逼近目标函数，并不保证梯度下降能找到它。可学性与可表示性是两件事。
- **“梯度消失全是 Sigmoid 的错”**——链式连乘、权重初始化过小、深层饱和都会导致梯度衰减。ReLU 缓解了饱和问题，但初始化与归一化同样关键。

八、要点回顾

1. 在感知器的输入层和输出层之间添加隐藏层，就可以得到多层感知器，异或这样的线性不可分问题随之可解。
2. 多层感知器是一类前馈神经网络，采用的是反向传播的学习方式，信号前向流动、误差反向流动。
3. 多输出情形下误差要写成平方和，否则正负误差会相互抵消，掩盖分类错误。
4. 反向传播算法要根据误差函数的梯度来调整权重系数，需要应用求导的链式法则。
5. 传递函数必须处处可微，符号函数因此在多层网络中退场，Sigmoid 成为标准选择。
6. 局域梯度的反向递推是算法的核心：输出层由预测与标签的差算出，隐藏层由上一层局域梯度的加权和算出。
7. 隐藏层的作用是特征检测，它把输入变换到新的特征空间，使原本不可分的问题变得可分。
8. 权重总数决定所需样本量，经验法则是样本数约为权重数的 10 倍；层数过多不会提升性能，反而增加训练难度。
9. 单个隐藏层就能使多层感知器以任意精度逼近任意连续函数，但该定理只保证表达力，不保证可训练性与泛化能力。
10. 早停与正则化都能抑制过拟合，前者从迭代次数入手，后者从损失函数入手，两者在效果上有相通之处。

九、名词汇总

名称	英文全称	英文缩写	简短介绍
多层感知器	Multilayer Perceptron	MLP	含一个或多个隐藏层的前馈神经网络

名称	英文全称	英文缩写	简短介绍
隐藏层	Hidden Layer	—	位于输入层与输出层之间、不直接接触外部信号的层
全连接	Fully Connected	FC	每个神经元与上一层所有神经元相连的连接方式
前馈神经网络	Feedforward Neural Network	FNN	信号单向逐层前传、层间无回路的神经网络
反向传播	Backpropagation	BP	用链式法则从输出层向输入层逐层求梯度的算法
误差函数	Error Function	—	度量网络输出与真实标签差距的目标函数
平方误差	Squared Error	—	各输出误差平方求和，避免正负抵消
均方误差	Mean Squared Error	MSE	平方误差的均值形式，常用作回归任务的损失
传递函数	Transfer Function	—	神经元上把加权输入映射为输出的非线性函数
对数几率函数	Logistic Function	—	即 Sigmoid，把实数压缩到 (0,1) 且处处可导
链式法则	Chain Rule	—	复合函数求导时逐环相乘的法则
局域梯度	Local Gradient	δ	误差对某神经元净输入的偏导，反向逐层递推
学习率	Learning Rate	η	控制每次梯度更新步长的正系数
梯度下降	Gradient Descent	GD	沿负梯度方向迭代更新参数以最小化损失
对称性破缺	Symmetry Breaking	—	用随机初始化使同层神经元分化出不同功能
特征检测	Feature Detection	—	隐藏层把输入变换到新特征空间并识别突出特征
万能逼近定理	Universal Approximation Theorem	—	足够宽的单个隐藏层可任意精度逼近连续函数
早停	Early Stopping	—	验证误差开始上升时立即终止训练以抑制过拟合
正则化	Regularization	—	在损失上加入权重惩罚项以限制模型复杂度
过拟合	Overfitting	—	模型记住训练集噪声、在新样本上表现变差的现象

名称	英文全称	英文缩写	简短介绍
验证集	Validation Set	—	训练过程中用于监控泛化能力、不参与参数更新的数据
计算图	Computational Graph	—	把前向计算表示为节点与边、便于自动求导的结构
自动微分	Automatic Differentiation	AD	由程序自动应用链式法则计算梯度的技术